

Sopa de Letras con Resolución Automática

Documentación Técnica

Inteligencia Artificial

Índice

1. Descripción del Proyecto	2
2. Generación de la Sopa	2
2.1. Cargar palabras	2
2.2. Colocar palabras en el tablero	2
2.3. Rellenar espacios vacíos	2
3. Algoritmo de Búsqueda (Solver)	2
4. Resaltado y Animación	3
5. Detección Manual	3
6. Temporizador	3
7. Interfaz Gráfica	3

1. Descripción del Proyecto

Juego de **Sopa de Letras** desarrollado en Java con Swing y dark theme. Las palabras se cargan desde un archivo externo `palabras.txt`. Incluye resolución automática con animación, temporizador por palabra y generación dinámica de nuevas sopas.

2. Generación de la Sopa

2.1. Cargar palabras

Las palabras se leen del archivo `palabras.txt`, una por línea, y se almacenan en una lista. Se normalizan a mayúsculas y se eliminan tildes para simplificar la comparación.

2.2. Colocar palabras en el tablero

Para cada palabra se intentan posiciones y direcciones al azar hasta encontrar un espacio válido:

```
1 boolean colocarPalabra(String palabra) {
2     int intentos = 0;
3     while (intentos++ < 1000) {
4         int fila = rand.nextInt(FILAS);
5         int col = rand.nextInt(COLS);
6         int dir = rand.nextInt(8); // 8 direcciones
7
8         if (cabe(palabra, fila, col, dir) &&
9             noColisiona(palabra, fila, col, dir)) {
10             escribir(palabra, fila, col, dir);
11             return true;
12         }
13     }
14     return false; // no pudo colocarse
15 }
```

Las 8 direcciones son: derecha, izquierda, abajo, arriba y las 4 diagonales.

2.3. Rellenar espacios vacíos

Después de colocar todas las palabras, las celdas vacías se llenan con letras aleatorias del abecedario español para ocultar las palabras.

3. Algoritmo de Búsqueda (Solver)

El solver busca cada palabra verificando todas las celdas del tablero como posible punto de inicio:

```
1 int[] buscarPalabra(String palabra) {
2     int[][] dirs = {{0,1},{0,-1},{1,0},{-1,0},
3                     {1,1},{1,-1},{-1,1},{-1,-1}};
4     for (int f = 0; f < FILAS; f++) {
```

```

5         for (int c = 0; c < COLS; c++) {
6             for (int[] dir : dirs) {
7                 if (coincide(palabra, f, c, dir))
8                     return new int[]{f, c,
9                                     dir[0], dir[1]};
10            }
11        }
12    }
13    return null;
14 }

```

La función `coincide()` compara carácter por carácter en la dirección indicada, verificando que no salga del tablero.

4. Resultado y Animación

Cuando se encuentra una palabra (manual o automáticamente):

- Se calculan todas las celdas que ocupa.
- Se pintan en color destacado usando `paintComponent()`.
- La palabra se marca como encontrada en la lista lateral.
- Para **Resolver Todo**, se usa un `javax.swing.Timer` con 300ms entre palabras para crear un efecto de animación.

5. Detección Manual

El jugador hace clic en la primera letra de la palabra y arrastra hasta la última. Se registran las coordenadas de inicio y fin del arrastre, y se valida si forman una línea recta en alguna de las 8 direcciones. Si la palabra formada coincide con alguna de la lista, se resalta automáticamente.

6. Temporizador

Cada palabra tiene un tiempo límite configurable. Un `javax.swing.Timer` decrementa el contador cada segundo y actualiza la etiqueta de tiempo. Si el tiempo llega a 0, se pasa automáticamente a la siguiente palabra.

7. Interfaz Gráfica

- **Panel de tablero:** dibujado con `paintComponent()`, fondo oscuro (`#1e1e1e`), letras en blanco.
- **Panel lateral:** lista de palabras a encontrar con estado visual (tachadas las encontradas).
- **Botones:** “Nueva Sopa”, “Siguiente Palabra”, “Resolver Todo”.

- Mouse listeners detectan inicio y fin del arrastre del jugador.